

Pontifícia Universidade Católica de Campinas
Escola Politécnica

Backend integrado ao Oracle Database

INTRODUÇÃO AS TECNOLOGIAS WEB

José Guilherme Picolo
E-mail: jose.picolo@puc-campinas.edu.br

Do array para uma tabela

Persistência é a conservação dos dados além da execução do processo

Antes: let alunos: Aluno[]

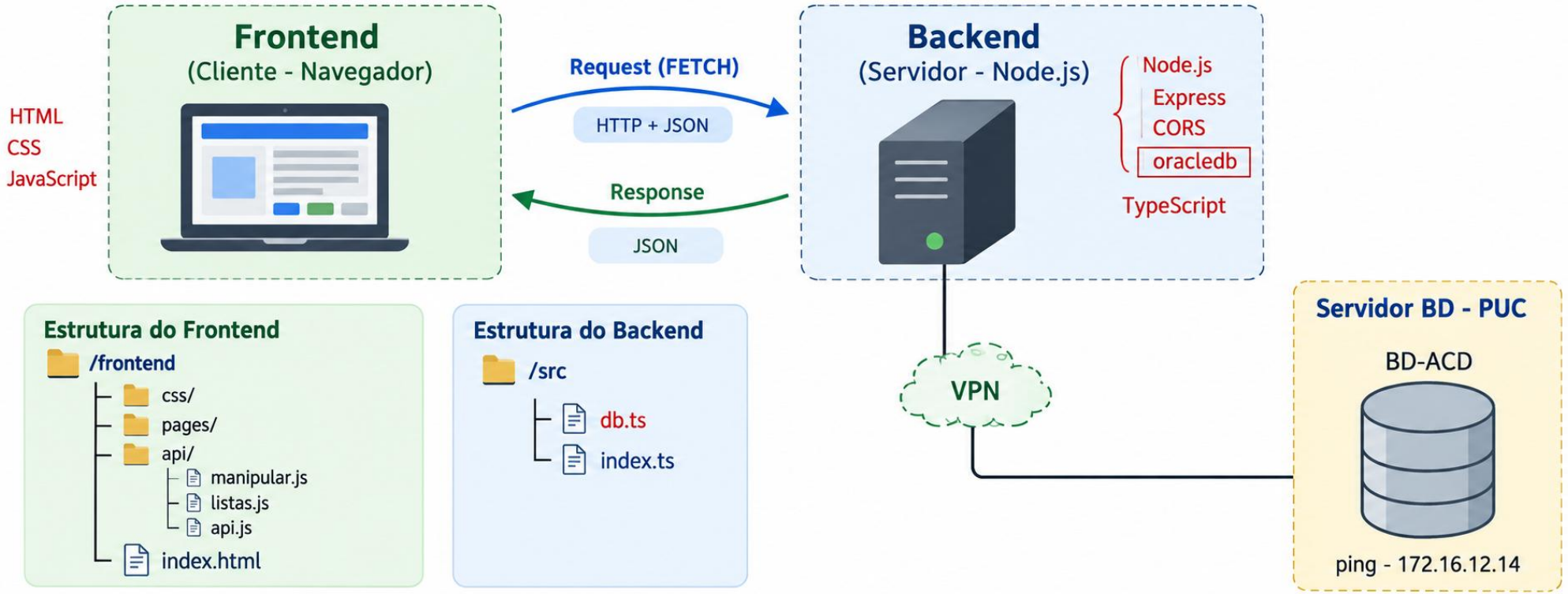
O processo Node.js guarda os objetos na memória. Ao reiniciar, o array volta à inicialização.

Agora: tabela alunos

O Oracle Database armazena os registros independentemente da execução do backend.

A arquitetura completa

HTTP comunica frontend e API. O driver oracledb comunica API e banco



O navegador chama a API. Usuário e senha do banco ficam no backend.

O que muda no código atual?

A interface HTTP permanece. A forma de acessar os dados muda

Código em memória	Versão com Oracle
let alunos: Aluno[] = [...]	Tabela alunos criada com CREATE TABLE
res.json(alunos)	SELECT e res.json(resultado.rows ?? [])
alunos.find(...)	SELECT ... WHERE ra = :ra
alunos.push(novoAluno)	INSERT com binds e autoCommit: true
Callback síncrono	Callback async com await e try/catch

Do objeto Aluno ao modelo relacional

Uma linha representa um aluno. Cada coluna representa um atributo

TypeScript	Oracle	Regra da aula
ra: number	NUMBER(10,0)	Chave primária, inteiro positivo
nome: string	VARCHAR2(120 CHAR)	Obrigatório e não vazio
email: string	VARCHAR2(180 CHAR)	Obrigatório e único
curso: CursoAluno	VARCHAR2(40 CHAR)	Um dos dois cursos permitidos
semestre: number	NUMBER(2,0)	Entre 1 e 10
situacao: SituacaoAluno	VARCHAR2(10 CHAR)	Quatro valores, padrão Ativo

Acesse a VPN para ter acesso ao BD-ACD

Utilize o tutorial disponível no CANVAS para se conectar a VPN e assim conseguir acessar o servidor da PUC.
O monitor da disciplina também fez um tutorial mais detalhado de como fazer a conexão que está disponível no TEAMS.



Criação da tabela alunos

Execute no seu schema Oracle, usando um cliente SQL.




```sql
CREATE TABLE alunos (
  ra NUMBER(10,0) PRIMARY KEY CHECK (ra > 0),
  nome VARCHAR2(120 CHAR) NOT NULL CHECK (TRIM(nome) IS NOT NULL),
  email VARCHAR2(180 CHAR) NOT NULL UNIQUE CHECK (TRIM(email) IS NOT NULL),
  curso VARCHAR2(40 CHAR) NOT NULL CHECK (
    curso IN ('Engenharia de Software', 'Sistemas de Informação')
  ),
  semestre NUMBER(2,0) NOT NULL CHECK (semestre BETWEEN 1 AND 12),
  situacao VARCHAR2(10 CHAR) DEFAULT 'Ativo' NOT NULL CHECK (
    situacao IN ('Ativo', 'Formado', 'Trancado', 'Inativo')
  )
);
```

O RA é informado no cadastro. O exemplo não usa sequência nem coluna IDENTITY.

As regras também existem no banco

A API valida a requisição. As constraints preservam a integridade dos registros

Regra SQL	Efeito
PRIMARY KEY	Impede RA duplicado e nulo
NOT NULL	Exige um valor na coluna
UNIQUE	Impede a repetição do e-mail
CHECK	Restringe valores, como o semestre de 1 a 12
DEFAULT 'Ativo'	Preenche a situação quando o INSERT omite a coluna

O tipo TypeScript não impede que um JSON inválido chegue à API. A validação ocorre em execução.

Dados iniciais e confirmação com COMMIT

Cada INSERT cria uma linha. COMMIT confirma as alterações

```sql
INSERT INTO alunos
  (ra, nome, email, curso, semestre, situacao)
VALUES (1, 'João da Silva', 'joao@aluno.edu.br',
        'Engenharia de Software', 2, 'Ativo');

INSERT INTO alunos
  (ra, nome, email, curso, semestre, situacao)
VALUES (2, 'Maria da Silva', 'maria@aluno.edu.br',
        'Sistemas de Informação', 8, 'Formado');

COMMIT;
SELECT * FROM alunos ORDER BY ra;
```

Confirme o INSERT para que outra conexão, como a do backend, possa enxergar os registros.

Dependências do backend

Execute os comandos na pasta em que está o package.json

```
npm install oracledb dotenv

npm install -D @types/oracledb
```

Express e cors já estão instalados no projeto atual.

oracledb

Driver oficial para acessar o Oracle Database.

dotenv

Carrega variáveis do arquivo .env para process.env.

@types/oracledb

Declarações de tipos utilizadas pelo TypeScript.

Configuração do arquivo .env

Crie o arquivo na raiz do backend, ao lado do package.json

```
#Banco da PUC, via rede/VPN institucional
ORACLE_HOST=172.16.12.14
ORACLE_PORT=1521
ORACLE_SERVICE=XE
ORACLE_USER=SEU_USUARIO
ORACLE_PASSWORD=SUA_SENHA
PORT=3000
```

```
# Alternativa: banco local
ORACLE_HOST=localhost
ORACLE_PORT=1521
ORACLE_SERVICE=XEPDB1
ORACLE_USER=SEU_USUARIO
ORACLE_PASSWORD=SUA_SENHA
PORT=3000
```

Inclua .env no .gitignore. Use apenas a configuração correspondente ao seu ambiente.
Usuário e Senha foram fornecidos pelo professor(a) de Estudos de Banco de Dados I.

db.ts: variáveis e formato dos resultados

A configuração fica centralizada no arquivo db.ts; copiar o arquivo para dentro da pasta /src junto ao index.ts do backend.

```
1  import oracledb from "oracledb";
2  import "dotenv/config";
3
4  const {
5    ORACLE_HOST, ORACLE_PORT, ORACLE_SERVICE,
6    ORACLE_USER, ORACLE_PASSWORD
7  } = process.env;
8
9  const connectString =
10   `${ORACLE_HOST}:${ORACLE_PORT}/${ORACLE_SERVICE}`;
11
12 oracledb.outFormat = oracledb.OUT_FORMAT_OBJECT;
```

.env

Separa as credenciais do código-fonte.

connectString

Neste formato, o trecho após / é o SERVICE_NAME.

outFormat

Configura os SELECTs para retornarem objetos JavaScript.

OUT_FORMAT_OBJECT não altera as letras dos campos: os aliases ainda serão necessários.

db.ts: função initPool()

O servidor aguarda a criação do pool antes de receber requisições

```
export async function initPool() {
  await oracledb.createPool({
    user: ORACLE_USER,
    password: ORACLE_PASSWORD,
    connectString,
    poolMin: 1,
    poolMax: 10,
    poolIncrement: 1
  });
}
```

poolMin: 1

Mínimo configurado para o pool.

poolMax: 10

Limita a quantidade de conexões simultâneas.

poolIncrement: 1

Define o crescimento do pool quando novas conexões forem necessárias.

Chame initPool() uma vez na inicialização.

db.ts: getConn() e closePool()

Uma função atende às rotas. A outra atende ao encerramento da aplicação

```
1  export async function getConn() {
2    return oracledb.getPool().getConnection();
3  }
4
5  export async function closePool() {
6    try {
7      await oracledb.getPool().close(10);
8    } catch {
9      /* pool pode não existir, ignore */
10   }
11 }
```

getConn()

Obtém uma conexão do pool padrão já criado.

conn.close()

Na rota, devolve essa conexão ao pool.

closePool()

Encerra o pool. O 10 é o tempo de drenagem, em segundos.

No código fornecido, o catch de closePool() silencia qualquer erro de fechamento.

O ciclo de vida de uma conexão

Uma consulta usa uma conexão do pool e depois a devolve

Função ou opção	Responsabilidade
oracledb.createPool()	Inicializa o pool compartilhado
oracledb.getConnection()	Obtém uma conexão do pool padrão
conn.execute(sql, binds, options)	Executa a instrução SQL
autoCommit: true	Confirma a alteração ao concluir a execução
OUT_FORMAT_OBJECT	Retorna linhas como objetos JavaScript
conn.close() em finally	Devolve a conexão ao pool, mesmo após erro

pool.getConnection() também pode obter uma conexão do pool retornado por createPool().

Alterações no início do index.ts

Novos imports, inicialização do servidor e leitura da porta

```
1 import "dotenv/config";
2 import { initPool, getConn, closePool } from
3 "./db";
4 const app = express();
5 app.use(express.json());
6 app.use(cors());
7 iniciarServidor();
8
9 const PORT = Number(process.env.PORT ?? 3000);
```

dotenv/config

Carrega as variáveis do arquivo .env.

Import de ./db

Disponibiliza as três funções do pool.

iniciarServidor()

Executa a função que prepara pool e servidor.

PORT

Lê a porta do ambiente ou usa 3000.

A chamada chega a await initPool(). Enquanto aguarda, o módulo continua e inicializa PORT.

index.ts - iniciarServidor(): função explícita

A aplicação prepara o pool antes de abrir a porta HTTP

```
async function iniciarServidor() {
    try {
        await initPool();
        app.listen(PORT, () => {
            console.log(`Servidor ativo na porta ${PORT}`);
        });
        process.on("SIGINT", encerrarServidor);
        process.on("SIGTERM", encerrarServidor);
    } catch (erro) {
        console.error("Falha ao iniciar o servidor:", erro);
        process.exit(1);
    }
}
```

SIGINT trata Ctrl+C. SIGTERM trata uma solicitação de término enviada pelo ambiente.

index.ts - encerrarServidor(): fechamento do pool

Os sinais registrados chamam esta função

```
async function encerrarServidor() {
    try {
        await closePool();
        console.log("Pool de conexões encerrado.");
        process.exit(0);
    } catch (erro) {
        console.error("Falha ao encerrar o pool:", erro);
        process.exit(1);
    }
}
```

await closePool()

Aguarda o encerramento do pool.

process.exit(0)

Encerra o processo indicando sucesso.

process.exit(1)

Encerra o processo indicando falha.

O padrão simples das rotas – Modelo Geral

Todas seguem obter conexão, executar SQL, responder e devolver a conexão

```
let conn;
try {
  conn = await getConn();
  const result = await conn.execute(sql, binds);
  return res.status(200).json(result.rows ?? []);
} catch (erro) {
  console.error(erro);
  return res.status(500).json({
    mensagem: "Erro na consulta!"
  });
} finally {
  if (conn) await conn.close();
}
```

finally executa também quando existe return ou quando execute lança um erro.

Aliases: nomes dos campos no JSON

OUT_FORMAT_OBJECT define objetos. AS com aspas define os nomes exatos

```
SELECT ra AS "ra", nome AS "nome", email AS "email",
       curso AS "curso", semestre AS "semestre",
       situacao AS "situacao"
FROM alunos

// Cada linha chega com estas propriedades:
// { ra, nome, email, curso, semestre, situacao }
```

> O frontend pode continuar acessando aluno.ra, aluno.nome e os demais campos em minúsculas.

GET /alunos/:ra: consulta individual

O RA da URL entra no bind :ra do SELECT

```
app.get("/alunos/:ra", async (req: Request, res: Response)
=> {const ra = Number(req.params.ra);
    let conn;
    try {
        conn = await getConn();
        const result = await conn.execute(
            `SELECT ra AS "ra", nome AS "nome", email AS
"email",
             curso AS "curso", semestre AS "semestre",
             situacao AS "situacao"
             FROM alunos WHERE ra = :ra`,
            { ra }
        );
```

req.params.ra

Lê o identificador informado no caminho.

Number(...)

Converte o valor do parâmetro para número.

{ ra }

Fornece o valor do bind :ra na consulta.

GET por RA: objeto ou null

Continuação da mesma rota, após executar o SELECT

```
        return res.status(200).json(result.rows?.[0] ?? null);
    } catch (erro) {
        console.error(erro);
        return res.status(500).json({
          mensagem: "Erro ao buscar aluno!"
        });
    } finally {
        if (conn) await conn.close();
    }
});
```

rows?.[0]

Obtém a primeira linha do resultado quando ela existe.

?? null

Define null como resposta quando não há aluno.

Status 200

A busca sem registro também responde 200 nesta versão.

O frontend deve conferir se o aluno retornado é null antes de preencher o formulário.

POST /alunos: execução do INSERT

O body fornece os dados. O SQL fixa a situação inicial como Ativo

```
app.post("/alunos", async (req: Request, res: Response) => {
    const { ra, nome, email, curso, semestre } = req.body;
    let conn;
    try {
        conn = await getConn();
        await conn.execute(
            `INSERT INTO alunos (ra, nome, email, curso, semestre, situacao)
             VALUES (:ra, :nome, :email, :curso, :semestre, 'Ativo')`,
            { ra, nome, email, curso, semestre },
            { autoCommit: true }
        );
```

Os valores seguem no objeto de binds. autoCommit: true confirma a gravação.

POST /alunos: mensagem de cadastro

Continuação da mesma rota, após o INSERT

```
        return res.status(201).json({
            mensagem: "Aluno cadastrado com sucesso!"
        });
    } catch (erro) {
        console.error(erro);
        return res.status(500).json({
          mensagem: "Erro ao cadastrar aluno!"
        });
    } finally {
        if (conn) await conn.close();
    }
});
```

Status 201

Indica a criação bem-sucedida do cadastro.

JSON de sucesso

Contém apenas a propriedade mensagem.

finally

Devolve a conexão depois da operação.

O POST não retorna o objeto aluno nesta versão. Para consultá-lo, use o GET por RA.

autoCommit: confirmação da gravação

INSERT e UPDATE recebem esta opção no terceiro argumento

```
await conn.execute(
  sql,
  binds,
  { autoCommit: true }
);
```

No script inicial, COMMIT confirma os INSERTs executados no cliente SQL.

SQL

A instrução que será executada.

Binds

Os valores usados nos marcadores.

autoCommit: true

Confirma a alteração quando o comando termina com sucesso.

PUT /alunos/:ra: execução do UPDATE

O RA da rota identifica a linha. O body fornece os novos valores

```
app.put("/alunos/:ra", async (req: Request, res: Response) => {
    const ra = Number(req.params.ra);
    const { nome, email, curso, semestre, situacao } = req.body;
    let conn;
    try {
        conn = await getConn();
        const result = await conn.execute(
            `UPDATE alunos
             SET nome = :nome,
                 email = :email,
                 curso = :curso,
                 semestre = :semestre,
                 situacao = :situacao
             WHERE ra = :ra`,
            { ra, nome, email, curso, semestre, situacao },
            { autoCommit: true }
        );
```

O UPDATE já fica confirmado por autoCommit: true antes da consulta apresentada no próximo slide.

PUT /alunos/:ra: retorno do aluno atualizado

Continuação: conferir rowsAffected e consultar os dados para o frontend

```
        if (result.rowsAffected === 0) {
            return res.status(404).json({ mensagem: "Aluno não encontrado!" });
        }
        const busca = await conn.execute(
            `SELECT ra AS "ra", nome AS "nome", email AS "email",
                    curso AS "curso", semestre AS "semestre",
                    situacao AS "situacao"
             FROM alunos WHERE ra = :ra`,
            { ra }
        );
        return res.status(200).json(busca.rows?.[0]);
    } catch (erro) {
        console.error(erro);
        return res.status(500).json({
          mensagem: "Erro ao atualizar aluno!"
        });
    } finally {
        if (conn) await conn.close();
    }
});
```

Nenhuma linha atualizada: 404. Atualização bem-sucedida: 200 com o objeto aluno consultado.

DELETE lógico: alteração para Inativo

A rota conserva o registro e altera apenas sua situação

```
app.delete("/alunos/:ra", async (req: Request, res: Response) => {
    const ra = Number(req.params.ra);
    let conn;
    try {
        conn = await getConn();
        const result = await conn.execute(
            `UPDATE alunos
             SET situacao = 'Inativo'
             WHERE ra = :ra`,
            { ra },
            { autoCommit: true }
        );
        if (result.rowsAffected === 0) {
            return res.status(404).json({ mensagem: "Aluno não encontrado!" });
        }
```

O endpoint usa DELETE, mas o comando SQL é UPDATE. A linha continua na tabela alunos.

DELETE lógico: retorno da mensagem e do aluno

Continuação: o SELECT recupera o cadastro já inativado

```
        const busca = await conn.execute(
            `SELECT ra AS "ra", nome AS "nome", email AS "email",
                    curso AS "curso", semestre AS "semestre",
                    situacao AS "situacao"
             FROM alunos WHERE ra = :ra`,
            { ra }
        );
        return res.status(200).json({
            mensagem: "Aluno inativado com sucesso!",
            aluno: busca.rows?.[0]
        });
    } catch (erro) {
        console.error(erro);
        return res.status(500).json({
          mensagem: "Erro ao inativar aluno!"
        });
    } finally {
        if (conn) await conn.close();
    }
});
```

O JSON contém mensagem e aluno. A propriedade aluno.situacao deve ser Inativo.

Contrato de respostas desta versão

Os testes e o frontend devem esperar exatamente estes formatos

Rota	Sucesso	Conteúdo do JSON
GET /alunos	200	Array de alunos ou []
GET /alunos/:ra	200	Objeto aluno ou null
POST /alunos	201	Objeto com mensagem
PUT /alunos/:ra	200	Objeto do aluno atualizado
DELETE /alunos/:ra	200	Objeto com mensagem e aluno

PUT e DELETE respondem 404 quando rowsAffected é 0. Erros capturados nas rotas respondem 500.

Problemas frequentes e diagnóstico

O status HTTP e o log do driver ajudam a localizar a falha

Sintoma	O que conferir
ORA-01017	Usuário e senha do schema
Falha de rede ou listener	VPN/rede, host, porta e nome do serviço
Banco incompatível com Thin	Versão do Oracle e necessidade de Thick/Client
ORA-00942	Tabela, schema e permissões do usuário
SELECT da API não mostra os dados	Mesmo schema e COMMIT no cliente SQL
A tela recebe RA e NOME	OUT_FORMAT_OBJECT e aliases com aspas duplas

Contrato de respostas desta versão

Os testes e o frontend devem esperar exatamente estes formatos

Rota	Sucesso	Conteúdo do JSON
GET /alunos	200	Array de alunos ou []
GET /alunos/:ra	200	Objeto aluno ou null
POST /alunos	201	Objeto com mensagem
PUT /alunos/:ra	200	Objeto do aluno atualizado
DELETE /alunos/:ra	200	Objeto com mensagem e aluno

PUT e DELETE respondem 404 quando rowsAffected é 0. Erros capturados nas rotas respondem 500.